LAPORAN PRAKTIKUM
JOBSHEET 07 - OVERLOADING DAN OVERRIDING



Disusun Oleh:

Nama	:	Mohammad Hafidz Rafi' Rabbani
NIM	:	254107020084
Kelas	:	2H
No. Absen	:	14
Repository	:	github.com/hafidzrafi/java-oop

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026

Daftar Isi

1. Tujuan 3

2. Praktikum 3

2.1. Percobaan 1: Overloading dan Overriding Perhitungan Gaji (Manager dan Staff) . 3

2.1.1. Pertanyaan 6

3. Tugas 8

3.1. Tugas 1: Implementasi Overloading pada Class Segitiga 8

3.2. Tugas 2: Implementasi Overriding dan Dynamic Method Dispatch (Manusia, Dosen, Mahasiswa) 10

1. Tujuan

- Memahami konsep dasar overloading dan overriding dalam paradigma pemrograman berorientasi objek (OOP).
- Mengidentifikasi perbedaan mendasar antara compile-time polymorphism (static binding / overloading) dan runtime polymorphism (dynamic binding / overriding).
- Mengimplementasikan method overloading dengan variasi jumlah arity dan tipe data parameter formal.
- Mengimplementasikan method overriding pada hierarki inheritance dengan mempertahankan kompatibilitas signature method.
- Menerapkan teknik Dynamic Method Dispatch menggunakan referensi superclass untuk mengeksekusi method overriding secara polimorfik.
- Menyimpan dan mendokumentasikan seluruh source code pekerjaan pada repositori GitHub resmi: https://github.com/hafidzrafi/java-oop.

2. Praktikum

2.1. Percobaan 1: Overloading dan Overriding Perhitungan Gaji (Manager dan Staff)

Pada percobaan pertama, diimplementasikan hierarki pewarisan yang melibatkan kelas induk `Karyawan` beserta dua kelas turunannya, yaitu `Manager` dan `Staff`. Praktikum ini mendemonstrasikan kombinasi antara **method overriding** dan **method overloading** dalam satu skenario pengelolaan penggajian karyawan:

- **Method Overriding:** Method `getGaji()` milik kelas induk `Karyawan` dioverride oleh `Manager` untuk mengakomodasi penambahan atribut tunjangan (`tunjangan + super.getGaji()`).
- **Method Overloading:** Pada kelas `Staff`, method `getGaji()` dideklarasikan ulang dengan variasi parameter formal: `getGaji()` tanpa parameter untuk gaji pokok berbasis golongan, dan `getGaji(int lembur, double gajiLembur)` untuk menghitung total kompensasi lembur.
- **Komposisi dan Agregasi:** Kelas `Manager` mengelola daftar staf bawahannya melalui atribut array objek `Staff staff[]`.

```
// Manager.java (Cuplikan Overriding & Agregasi Staff)
package jobsheet.jobsheet07.src.experiments;
```

```java
public class Manager extends Karyawan {
    private double tunjangan;
    private String bagian;
    private Staff staff[];

    public void setTunjangan(double tunjangan) { this.tunjangan = tunjangan; }
    public void setBagian(String bagian) { this.bagian = bagian; }
    public void setStaff(Staff[] staff) { this.staff = staff; }

    public void lihatInfo() {
        System.out.println("Bagian      : " + this.getBagian());
        System.out.println("NIP         : " + this.getNip());
        System.out.println("Nama        : " + this.getNama());
        System.out.println("Golongan    : " + this.getGolongan());
        System.out.println("Tunjangan   : " + this.getTunjangan());
        System.out.println("Gaji        : " + this.getGaji());
        System.out.println("----------------------");
        this.lihatStaff();
    }

    @Override
    public double getGaji() {
        return super.getGaji() + tunjangan;
    }
}
```

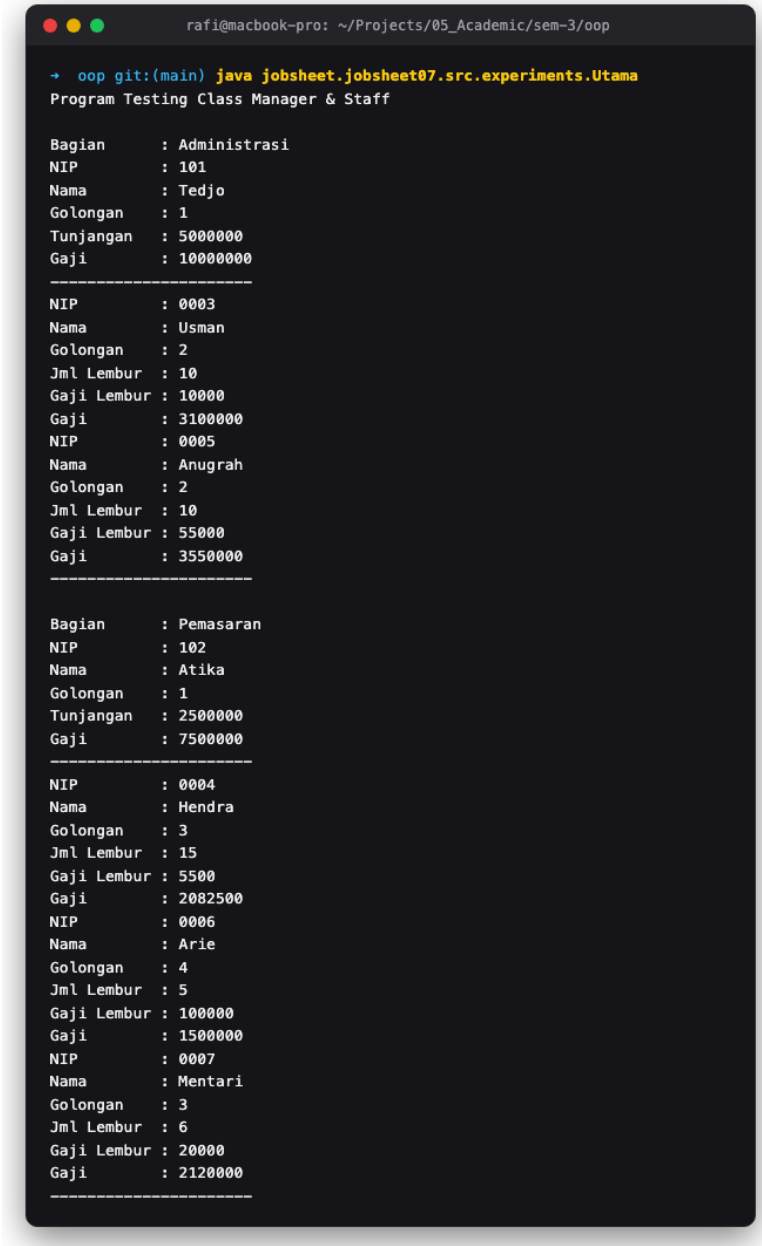
```java
// Staff.java (Cuplikan Method Overloading getGaji)
package jobsheet.jobsheet07.src.experiments;

public class Staff extends Karyawan {
    private int lembur;
    private double gajiLembur;

    public void setLembur(int lembur) { this.lembur = lembur; }
    public void setGajiLembur(double gajiLembur) { this.gajiLembur = gajiLembur; }

    // Method Overloading: getGaji dengan parameter lembur
    public double getGaji(int lembur, double gajiLembur) {
        return super.getGaji() + lembur * gajiLembur;
    }

    // Method Overriding: getGaji tanpa parameter memanggil versi overloaded
    @Override
    public double getGaji() {
        return super.getGaji() + lembur * gajiLembur;
    }
}
```


rafi@macbook-pro: ~/Projects/05_Academic/sem-3/oop
→ oop git:(main) java jobsheet.jobsheet07.src.experiments.Utama
Program Testing Class Manager & Staff
Bagian : Administrasi
NIP : 101
Nama : Tedjo
Golongan : 1
Tunjangan : 5000000
Gaji : 10000000
NIP : 0003
Nama : Usman
Golongan : 2
Jml Lembur : 10
Gaji Lembur : 10000
Gaji : 3100000
NIP : 0005
Nama : Anugrah
Golongan : 2
Jml Lembur : 10
Gaji Lembur : 55000
Gaji : 3550000
Bagian : Pemasaran
NIP : 102
Nama : Atika
Golongan : 1
Tunjangan : 2500000
Gaji : 7500000
NIP : 0004
Nama : Hendra
Golongan : 3
Jml Lembur : 15
Gaji Lembur : 5500
Gaji : 2082500
NIP : 0006
Nama : Arie
Golongan : 4
Jml Lembur : 5
Gaji Lembur : 100000
Gaji : 1500000
NIP : 0007
Nama : Mentari
Golongan : 3
Jml Lembur : 6
Gaji Lembur : 20000
Gaji : 2120000

2.1.1. Pertanyaan

1. From the source code above (Exercise 1 - PerkalianKu), where is the overloading?
    - Overloading terjadi pada deklarasi method `perkalian(int a, int b)` dan `perkalian(int a, int b, int c)` di dalam kelas `PerkalianKu`. Keduanya memiliki nama identifier yang identik dalam satu scope kelas, namun berbeda pada jumlah parameter formal (arity 2 parameter berbanding 3 parameter).
2. If there is overloading, how many different parameters are there?
    - Terdapat selisih 1 parameter pada jumlah arity. Method pertama menerima 2 parameter integer dengan type descriptor `(II)V`, sedangkan method kedua menerima 3 parameter integer dengan descriptor `(III)V`.
3. From the source code above (Exercise 2 - PerkalianKu with double), where is the overloading?
    - Overloading terjadi antara method `perkalian(int a, int b)` dan `perkalian(double a, double b)` di dalam kelas `PerkalianKu`. Keduanya memiliki nama method dan arity parameter yang sama (masing-masing 2 parameter), namun tipe data dari parameter formalnya berbeda.
4. If there is overloading, how many different types of parameters are there?
    - Terdapat 2 jenis tipe data parameter yang berbeda, yaitu tipe data primitif 32-bit integer (`int`) dan tipe data primitif 64-bit IEEE 754 floating-point (`double`).
5. From the source coding above (Exercise 3 - Fish), where is overriding?
    - Overriding terjadi pada method `swim()` di dalam subclass `Piranha` yang mendefinisikan ulang method `swim()` milik superclass `Ikan`. Keduanya memiliki identifier nama (`swim`), return type (`void`), dan daftar parameter `()` yang sama persis.
6. Describe the source coding above if there is overriding?
    - Subclass `Piranha` mewarisi method `swim()` dari superclass `Ikan`, lalu mendefinisikan implementasi spesifik sendiri. Pada class `Fish`, variabel referensi `Ikan b = new Piranha();` dideklarasikan dengan tipe statis `Ikan`, namun menunjuk ke instance objek konkret `Piranha` di heap memory.

Ketika method `b.swim()` dipanggil, runtime JVM memanfaatkan tabel metode virtual (**vtable**) untuk melakukan dynamic method dispatch ke method overriding milik `Piranha`, sehingga teks yang tercetak adalah `"Piranha bisa makan daging"` dan bukan teks milik kelas `Ikan`.

3. Tugas

3.1. Tugas 1: Implementasi Overloading pada Class Segitiga

Tugas mandiri pertama mengimplementasikan konsep **compile-time polymorphism** (method overloading) pada kelas `Segitiga` sesuai diagram kelas UML:

- Atribut private `-sudut: int` untuk menyimpan kalkulasi sudut ketiga.
- Method `totalSudut(int sudutA): int` menghitung sudut ketiga bila diketahui satu sudut ($180^\circ - \text{sudutA}$).
- Method `totalSudut(int sudutA, int sudutB): int` menghitung sudut ketiga bila diketahui dua sudut ($180^\circ - (\text{sudutA} + \text{sudutB})$).
- Method `keliling(int sisiA, int sisiB, int sisiC): int` menghitung keliling segitiga sembarang ($a + b + c$).
- Method `keliling(int sisiA, int sisiB): double` menghitung keliling segitiga siku-siku dengan menghitung sisi miring hipotenusa menggunakan teorema Pythagoras ($a + b + \sqrt{a^2 + b^2}$).

```java
// Segitiga.java
package jobsheet.jobsheet07.src.assignments;

public class Segitiga {
    private int sudut;

    public int totalSudut(int sudutA) {
        this.sudut = 180 - sudutA;
        return this.sudut;
    }

    public int totalSudut(int sudutA, int sudutB) {
        this.sudut = 180 - (sudutA + sudutB);
        return this.sudut;
    }

    public int keliling(int sisiA, int sisiB, int sisiC) {
        return sisiA + sisiB + sisiC;
    }

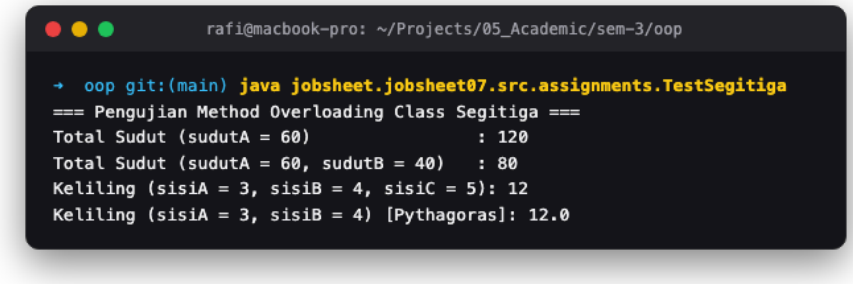
    public double keliling(int sisiA, int sisiB) {
        return sisiA + sisiB + Math.hypot(sisiA, sisiB);
    }
}
```

```java
// TestSegitiga.java
package jobsheet.jobsheet07.src.assignments;

public class TestSegitiga {
    public static void main(String[] args) {
        Segitiga s = new Segitiga();

        System.out.println("=== Pengujian Method Overloading Class Segitiga ===");
        System.out.println("Total Sudut (sudutA = 60)                : " +
s.totalSudut(60));
        System.out.println("Total Sudut (sudutA = 60, sudutB = 40)   : " +
s.totalSudut(60, 40));
        System.out.println("Keliling (sisiA = 3, sisiB = 4, sisiC = 5): " +
s.keliling(3, 4, 5));
        System.out.println("Keliling (sisiA = 3, sisiB = 4) [Pythagoras]: " +
s.keliling(3, 4));
    }
}
```

First-Principles Method Overloading: Resolusi method overloading diselesaikan secara statis oleh kompilator Java (**javac**) pada fase compile-time. Kompilator menentukan secara pasti method mana yang dituju berdasarkan method descriptor (jumlah dan tipe argumen yang di-push ke operand stack). Tidak ada overhead pencarian dinamis saat runtime, sehingga eksekusi langsung diarahkan ke label fungsi yang bersangkutan.



3.2. Tugas 2: Implementasi Overriding dan Dynamic Method Dispatch (Manusia, Dosen, Mahasiswa)

Tugas mandiri kedua mengimplementasikan hierarki kelas dan teknik **Dynamic Method Dispatch** (runtime polymorphism) berbasis diagram kelas UML:

- Superclass `Manusia` memiliki method umum `bernapas()` dan `makan()`.
- Subclass `Dosen` mewarisi `Manusia`, melakukan **overriding** pada method `makan()`, serta menambahkan method unik `lembur()`.
- Subclass `Mahasiswa` mewarisi `Manusia`, melakukan **overriding** pada method `makan()`, serta menambahkan method unik `tidur()`.

```java
// Manusia.java
package jobsheet.jobsheet07.src.assignments;

public class Manusia {
    public void bernapas() {
        System.out.println("Sedang bernapas");
    }

    public void makan() {
        System.out.println("Sedang makan");
    }
}
```

```java
// Dosen.java
package jobsheet.jobsheet07.src.assignments;

public class Dosen extends Manusia {
    @Override
    public void makan() {
        System.out.println("Dosen Sedang makan");
    }

    public void lembur() {
        System.out.println("Dosen sedang lembur");
    }
}
```

```java
// Mahasiswa.java
package jobsheet.jobsheet07.src.assignments;

public class Mahasiswa extends Manusia {
    @Override
    public void makan() {
        System.out.println("Mahasiswa sedang makan");
    }
```

```
    public void tidur() {
        System.out.println("Mahasiswa sedang tidur");
    }
}
```

```
// Main.java (Pengujian Dynamic Method Dispatch)
package jobsheet.jobsheet07.src.assignments;

public class Main {
    public static void main(String[] args) {
        System.out.println("=== Pengujian Langsung Subclass (Direct Instantiation)
===");
        Mahasiswa mahasiswa = new Mahasiswa();
        Dosen dosen = new Dosen();

        mahasiswa.makan();
        mahasiswa.bernapas();
        mahasiswa.tidur();

        System.out.println();
        dosen.makan();
        dosen.bernapas();
        dosen.lembur();

        System.out.println("\n=== Dynamic Method Dispatch (Runtime Polymorphism) ===");
        Manusia manusia1 = new Mahasiswa();
        Manusia manusia2 = new Dosen();

        manusia1.makan();
        manusia1.bernapas();

        System.out.println();
        manusia2.makan();
        manusia2.bernapas();
    }
}
```

Mekanisme Runtime Dynamic Method Dispatch (First Principles):

1. **Static vs Dynamic Type:** Pada deklarasi `Manusia manusia1 = new Mahasiswa();`, identifier `manusia1` memiliki tipe statis `Manusia` di tabel simbol kompilator, tetapi nilai alamat di stack frame merujuk pada instance objek `Mahasiswa` yang dialokasikan di heap memory.
2. **Opcode invokevirtual:** Saat baris `manusia1.makan()` dieksekusi, JVM memanggil opcode `invokevirtual`. Berbeda dengan static binding, `invokevirtual` membaca alamat Klass Word pada **Object Header** di heap

runtime guna menginspeksi **Virtual Method Table (vtable)** milik objek konkret `Mahasiswa`.

3. **Resolusi Dinamis:** Karena pointer vtable yang diperiksa menunjuk pada method table kelas `Mahasiswa`, JVM secara dinamis mendispatch eksekusi ke implementasi overriding `Mahasiswa.makan()`, menghasilkan output `"Mahasiswa sedang makan"`.
4. **Pewarisan Utuh Method Tanpa Overriding:** Method `bernapas()` tidak dioverride oleh `Mahasiswa` maupun `Dosen`. Maka pointer entry vtable untuk method `bernapas()` tetap merujuk langsung ke implementasi dasar pada kelas induk `Manusia`, mencetak `"Sedang bernapas"`.

